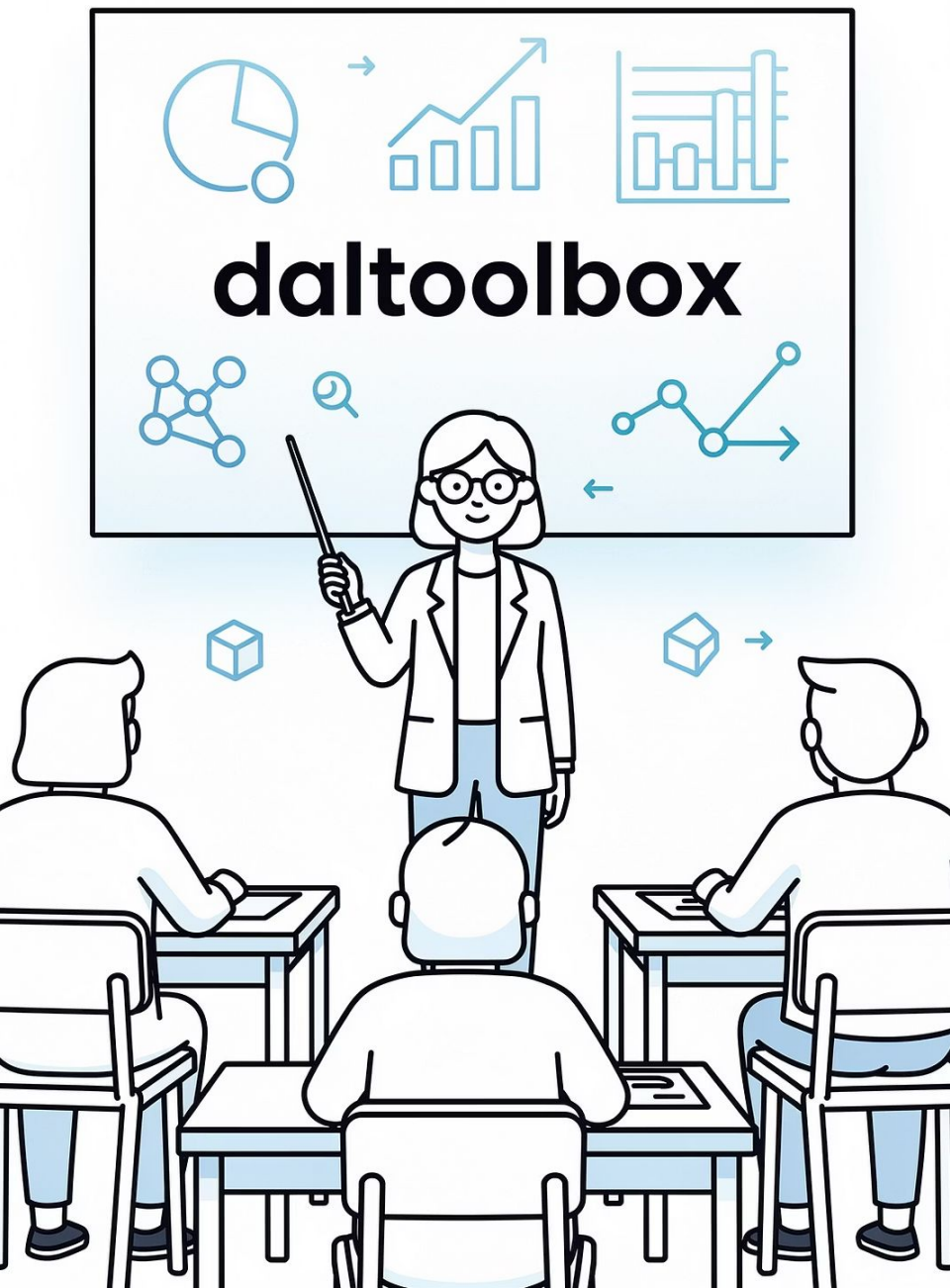




R Machine Learning Tutorial: Split, Fit, Evaluate, and Compare Models

Recommended Reading Order

The sequence is cumulative — each tutorial introduces a main objective and explains why that step matters in a data mining project.

01

01–04: Fundamentals

First experiment, sampling strategy, data quality, and preprocessing.

03

09–11: Pipelines

Complete pipeline for classification, regression, and clustering.

02

05–08: Modeling

Baseline, metrics, model comparison, and hyperparameter tuning.

04

12–13: Advanced

Visual analysis, reports, and custom framework extension.

The Supervised Learning Cycle

Supervised learning requires a disciplined experimental protocol to produce results that are both valid and reproducible. The core cycle — split, fit, predict, evaluate — is the backbone of every experiment in this tutorial series.

Split

Separate data into training and test sets before any modeling. The test set must never influence the learning process.

Fit

Train the model exclusively on the training set. The learner adjusts its internal parameters to the observed patterns.

Predict

Apply the trained model to new data. The model generalizes what it learned to unseen observations.

Evaluate

Measure performance on the test set. Compare training and test results to detect overfitting or underfitting.

  Reference: Mitchell, T. M. (1997). Machine Learning. McGraw-Hill. — Chapter 1 introduces the fundamental concepts of learning from data and generalization.

First Experiment

```
library(daltoolbox)

iris <- datasets::iris
slevels <- levels(iris$Species)

set.seed(1)
sr <- train_test(sample_random(), iris)

model <- cla_dtree("Species", slevels)
model <- fit(model, sr$train)

train_pred <- predict(model, sr$train)
train_eval <- evaluate(model,
  adjust_class_label(sr$train$Species), train_pred)
train_eval$metrics # accuracy=0.975

test_pred <- predict(model, sr$test)
test_eval <- evaluate(model,
  adjust_class_label(sr$test$Species), test_pred)
test_eval$metrics # accuracy=0.967
```

Step 1 — Load and identify classes: `levels()` returns the possible labels. The learner needs to know them before training.

Step 2 — Split: `sample_random()` + `train_test()` splits the data. Training teaches; testing evaluates on never-before-seen cases.

Step 3 — Fit: `cla_dtree()` instantiates the decision tree. `fit()` trains the model on the training data.

Step 4 — Evaluate: Always compare train vs. test. If train >> test, the model may be memorizing.

□ This cycle — split → fit → predict → evaluate — is the foundation of all the following tutorials.

Sampling Strategies

How data is divided between training and test sets directly affects the validity of any experimental conclusion. A biased split can make a weak model look strong — or a strong model look weak.

Random Sampling

Observations are assigned to training/test sets at random. Simple and fast, but may produce imbalanced class distributions in small datasets.

Stratified Sampling

Each class is sampled proportionally, guaranteeing that training and test sets reflect the original class distribution. Essential for imbalanced datasets.

k-Fold Cross-Validation

The dataset is divided into k equal parts. Each fold serves as the test set once, while the remaining $k-1$ folds are used for training. Produces a more robust performance estimate.

  Reference: Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. IJCAI, 14(2), 1137–1145. — Foundational study comparing sampling strategies for model evaluation.

Sampling Strategy

```
library(daltoolbox)

set.seed(1)
tt <- train_test(sample_stratified("Species"), iris)
table(tt$train$Species) # setosa=40, versicolor=40, virginica=40
table(tt$test$Species)  # setosa=10, versicolor=10, virginica=10

folds <- k_fold(sample_stratified("Species"), iris, 4)
# Cada fold: ~13/13/13 ou 12/12/12 por classe

slevels <- levels(iris$Species)
model <- cla_majority("Species", slevels)
model <- fit(model, tt$train)
eval <- evaluate(model, adjust_class_label(tt$test$Species),
  predict(model, tt$test))
eval$metrics # accuracy=0.333 — referência mínima
```

Why stratify?: `sample_random()` can produce imbalanced splits. `sample_stratified()` ensures each class appears in the same proportion in both training and test sets.

k_fold: Creates multiple subsets for cross-validation, each preserving the original class proportions.

Baseline on the split: Training `cla_majority()` on the stratified split reveals the performance floor — any useful model must surpass 0.333.

Expected result: Training with 40 samples per class; test with 10 per class.

📌 **Key concept:** Before comparing models, ensure the partition is not introducing bias. A poor split invalidates any comparison.

Data Quality: Missing Values, Outliers & Encoding

Real-world datasets are rarely clean. Addressing data quality issues before modeling is not optional — it directly determines whether the learned patterns reflect reality or artifacts of the data collection process.

Missing Values

Incomplete records introduce bias if ignored. Strategies range from row removal (`na.omit`) to imputation. The choice depends on the proportion of missing data and the mechanism behind it (MCAR, MAR, MNAR).

Outliers

Extreme observations can distort model parameters, especially in distance-based and linear methods. Detection via boxplot criterion ($1.5 \times \text{IQR}$) is a robust, distribution-free approach.

Categorical Encoding

Most learners require numeric inputs. One-hot encoding (dummy variables) converts each category into a binary column, preserving all information without imposing an artificial ordinal relationship.

📖 Reference: García, S., Luengo, J. & Herrera, F. (2015). Data Preprocessing in Data Mining. Springer. — Comprehensive treatment of missing data, outlier handling, and feature transformation techniques.

Data Quality and Cleaning

```
library(daltoolbox)

small <- iris[1:12, ]
small$Sepal.Length[c(3, 8)] <- NA
small_complete <- na.omit(small) # 9 linhas restantes

out      <- outliers_boxplot()
out      <- fit(out, iris)
outliers_found <- transform(out, iris)

cat_data  <- data.frame(
  color = factor(c("red", "blue", "green", "red")),
  value = c(10, 20, 15, 12))
mapper    <- categ_mapping("color")
cat_encoded <- transform(mapper, cat_data)
# colorblue colorgreen colorred
#      0      0      1 ...
```

Missing values: `na.omit()` removes rows with NA. It is the simplest strategy — useful as a cleaning baseline before attempting imputation.

Outliers: `outliers_boxplot()` detects suspicious observations using the boxplot criterion ($1.5 \times \text{IQR}$). The `fit()` + `transform()` pattern is the same as the rest of the framework.

Categorical variables: `categ_mapping()` applies one-hot encoding — each category becomes a binary column. Required for learners that expect numeric inputs.

📌 **Key concept:** Data quality decisions shape the experiment before the learner sees any data. Do not skip this step.

Normalization and Feature Selection

Raw features rarely arrive in a form that is ideal for learning. Normalization adjusts scale; feature selection reduces dimensionality. Both steps improve model performance and interpretability — but must be applied carefully to avoid data leakage.

Min-Max Normalization

Rescales each attribute to $[0, 1]$ using its observed minimum and maximum. Preserves the shape of the distribution and is essential for distance-sensitive methods (KNN, SVM, neural networks).

Z-Score Standardization

Centers each attribute at zero with unit variance. Preferred when the data follows a roughly Gaussian distribution or when the algorithm assumes zero-mean inputs.

Correlation-Based Selection

Removes attributes that are highly correlated with others ($r > \text{cutoff}$). Redundant features add noise without adding information, increasing training time and risk of overfitting.

Fit on Training Only

Normalization parameters (min, max, mean, std) must be learned from the training set alone. Applying them to the test set without refitting prevents data leakage.

📖 Reference: Guyon, I. & Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 1157–1182. — Foundational survey on feature selection methods, including correlation-based filtering and redundancy removal.

Basic Preprocessing

```
library(daltoolbox)

iris <- datasets::iris

norm  <- minmax()
norm  <- fit(norm, iris)
iris_norm <- transform(norm, iris)
# all numeric attributes in [0, 1]

fs  <- feature_selection_corr(cutoff = 0.9)
fs  <- fit(fs, iris_norm)
iris_fs <- transform(fs, iris_norm)

names(iris_norm)
# Sepal.Length Sepal.Width Petal.Length Petal.Width Species
names(iris_fs)
# Species Sepal.Length Sepal.Width Petal.Width
# Petal.Length removed due to high correlation
```

minmax(): Scales each numeric attribute to [0, 1] using its minimum and maximum. Essential for distance-sensitive methods such as KNN and SVM.

feature_selection_corr(): Removes attributes with correlation above the cutoff. In iris, `Petal.Length` is removed for being highly correlated with `Petal.Width` ($r > 0.9$).

fit/transform pattern: Both operations follow the same framework pattern — the object is fitted on the data and then applied. This makes it easy to apply the same transformation to the test set.

Result: Dataset goes from 5 to 4 columns without significant information loss.

❏ **Key concept:** Normalization prepares the scale; correlation-based selection prepares the structure. Both should be fitted on training data only.

Baseline Models in Classification

Before evaluating any sophisticated classifier, it is essential to establish a performance floor. A baseline model answers the question: "What accuracy can we achieve without actually learning anything useful?" Any model that fails to beat the baseline is not worth deploying.

Majority Class Baseline

Always predicts the most frequent class in the training set. For a balanced 3-class problem, this yields $1/3$ accuracy (≈ 0.333). Simple, transparent, and impossible to beat by chance.

Why Baselines Matter

Without a reference floor, it is impossible to know whether a model's accuracy is meaningful. A model with 70% accuracy on a dataset where the majority class is 90% is actually worse than doing nothing.

Experimental Fairness

The baseline must be trained and evaluated on exactly the same split as the competing models. Any difference in the split invalidates the comparison.

 Reference: Japkowicz, N. & Shah, M. (2011). Evaluating Learning Algorithms: A Classification Perspective. Cambridge University Press. — Chapter 3 discusses the role of baselines and naive classifiers in fair experimental design.

Classification Baseline

```
library(daltoolbox)

iris <- datasets::iris
slevels <- levels(iris$Species)
set.seed(1)
sr <- train_test(sample_stratified("Species"), iris)

baseline <- cla_majority("Species", slevels)
baseline <- fit(baseline, sr$train)
# accuracy=0.333 — intentional minimum reference

tree <- cla_dtree("Species", slevels)
tree <- fit(tree, sr$train)
tree_eval <- evaluate(tree,
  adjust_class_label(sr$test$Species),
  predict(tree, sr$test))
tree_eval$metrics # accuracy=0.967
```

cla_majority(): Always predicts the most frequent class in the training set. It is intentionally weak — it serves as the minimum comparison floor.

Why 0.333?: With 3 balanced classes, always predicting the same class yields 1/3 accuracy. Any useful model must surpass this.

Same split: Both models use exactly the same training and test data. This makes the comparison fair and any difference interpretable.

Result: The decision tree achieves 0.967 — a clear improvement over the baseline of 0.333.

❏ **Key concept:** Always compare an elaborate model against a transparent baseline — not just against other elaborate models.

Classification Metrics and Evaluation

A single number rarely tells the full story of a model's behavior. Understanding what each metric measures — and what it hides — is essential for making sound modeling decisions.

Accuracy

Proportion of correct predictions over all predictions. Intuitive but misleading on imbalanced datasets — a model that always predicts the majority class can achieve high accuracy while being useless.

Confusion Matrix

A table showing predicted vs. observed classes. Reveals exactly where the model makes mistakes: which classes are confused with which. Essential for multi-class problems.

Overfitting Detection

Compare training accuracy vs. test accuracy. If training $\gg$ test, the model memorized the training data instead of learning generalizable patterns.

Precision & Recall

Precision measures how many predicted positives are truly positive; recall measures how many actual positives were found. The F1-score balances both when classes are imbalanced.

 Reference: Sokolova, M. & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks.

Information Processing & Management, 45(4), 427–437. — Comprehensive review of accuracy, precision, recall, F1, and confusion-matrix-based metrics for multi-class classification.

Metrics and Evaluation

Connecting the numbers to model behavior is essential for real interpretation.

```
library(daltoolbox)

# ... (setup: iris, slevels, split, fit do cla_dtree) ...

train_eval <- evaluate(model,
  adjust_class_label(sr$train$Species),
  predict(model, sr$train))
train_eval$metrics # accuracy=0.975

test_eval <- evaluate(model,
  adjust_class_label(sr$test$Species),
  predict(model, sr$test))
test_eval$metrics # accuracy=0.967

table(predicted = test_eval$prediction,
  observed = sr$test$Species)
#      setosa versicolor virginica
# setosa      11       0       0
# versicolor   0      12       1
# virginica    0       0       6
```

- **Training metrics:** The model has already seen this data — the numbers tend to be optimistic. Training accuracy=0.975 looks good, but it is not sufficient on its own.
- **Test metrics:** This is the number that matters. It measures performance on never-before-seen data. Here: 0.967 — close to training, a sign of good generalization.
- **Confusion matrix:** Goes beyond accuracy. It shows exactly where the model makes mistakes — in this case, 1 virginica misclassified as versicolor.
- **Practical rule:** Training >> Test indicates overfitting. Training $\approx$ Test indicates good generalization.

📌 **Key concept:** Read training and test together. A single accuracy number does not tell the complete story.

Comparing Classifiers Fairly

Comparing models is only meaningful when the experimental conditions are identical. Changing the dataset, the split, or the metrics between models introduces confounds that make it impossible to attribute performance differences to the learner itself.

Fixed Experimental Design

The dataset, the train/test split, and the evaluation metrics must remain constant across all models. Only the learner changes. This is the minimum condition for a fair comparison.

Comparing Across Families

Comparing models from different algorithmic families (e.g., decision tree vs. KNN vs. ensemble) reveals structural differences in how each method partitions the feature space. Comparing variants of the same model is less informative.

Statistical Significance

On small datasets, performance differences may be due to chance. Repeated experiments with different random seeds, or statistical tests (e.g., Wilcoxon signed-rank), are needed to confirm that one model is genuinely better.

  Reference: Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1–30. — Widely cited guide on how to compare classifiers rigorously using non-parametric statistical tests.

Model Comparison

The central idea is **experimental fairness**: keeping the dataset, the split, and the metrics constant to evaluate different models.

```
library(daltoolbox)

# ... (setup: iris, slevels, stratified split) ...

models <- list(
  majority = cla_majority("Species", slevels),
  tree     = cla_dtree("Species", slevels),
  knn      = cla_knn("Species", slevels, k = 5),
  rf       = cla_rf("Species", slevels, mtry = 2, ntree = 10)
)

results <- lapply(models, function(m) {
  fitted <- fit(m, iris_train)
  pred   <- predict(fitted, iris_test)
  evaluate(fitted,
    adjust_class_label(iris_test$Species), pred)$metrics
})
# majority: accuracy=0.333
# tree/knn/rf: accuracy=0.967
```

- **Experimental fairness**: All models use the same split, the same dataset, and the same metrics. Only the learner changes — any difference in results is attributable to the model.
- **Different families**: majority (baseline), tree (interpretable), knn (local), rf (ensemble). Comparing distinct families reveals patterns that comparing variants of the same model does not.
- **lapply()**: Iterates over the list applying the same protocol to each model. The experimental design remains stable while the learner varies.
- **Result**: tree, knn, and rf tie at 0.967 — on iris, the problem is simple enough that several methods perform well.

❏ **Key concept**: The learner changes; the experimental design remains stable. That is the discipline of a fair comparison.

Hyperparameter Tuning

Most learning algorithms have parameters that are not learned from data — they must be set before training begins. Tuning these hyperparameters systematically can significantly improve model performance, but only when the experimental fundamentals are already solid.

What Are Hyperparameters?

Configuration values set before training: learning rate, number of neighbors (k), kernel type, regularization strength. Unlike model parameters, they are not updated during fitting.

Avoiding Leakage in Tuning

Hyperparameter selection must use only training data (e.g., via cross-validation on the training set). Using the test set to select hyperparameters inflates performance estimates and invalidates the evaluation.

Grid Search

Exhaustively evaluates all combinations of hyperparameter values in a predefined grid. Guaranteed to find the best combination within the grid, but computationally expensive for large search spaces.

Tuning vs. Complexity

More hyperparameters mean more combinations to evaluate. Start with a small, interpretable grid. Expand only if the initial results suggest that the search space is too narrow.

  Reference: Hutter, F., Kotthoff, L. & Vanschoren, J. (eds.) (2019). Automated Machine Learning. Springer. — Chapter 1 covers hyperparameter optimization fundamentals, including grid search, its guarantees, and its computational cost.

Tuning Workflow

Tuning is a disciplined extension of the workflow, applied only after the experimental fundamentals are under control.

```
library(daltoolbox)

# ... (setup: iris, slevels, random split) ...

tune <- cla_tune(
  cla_svm("Species", slevels),
  ranges = list(
    epsilon = seq(0, 0.4, 0.2),
    cost    = seq(20, 60, 20),
    kernel  = c("linear", "radial")
  )
)
model <- fit(tune, iris_train)
# returns the best model found

train_eval$metrics # accuracy=0.983
test_eval$metrics  # accuracy=0.967
```

- **cla_tune()**: Wraps any learner in a hyperparameter search. The search space is defined by `ranges` — here, 3 epsilon values × 3 cost values × 2 kernels = 18 combinations.
- **fit() on the tune object**: Tests all combinations and returns the best-performing model. The interface is identical to any other learner.
- **Compact space**: The example is intentionally small so that the tuning structure is understandable without lengthy computations.
- **Result**: Marginal improvement on training (0.975 → 0.983); test stable at 0.967. On iris, the gain is small — on more complex datasets, tuning can be decisive.

📌 **Key concept:** Tuning refines a solid workflow. There is no point in tuning a model on a poor split or without a reference baseline.

Data Leakage and Pipeline Integrity

Data leakage occurs when information from outside the training set is used to build the model. It is one of the most common and damaging mistakes in applied machine learning — producing models that appear to perform well in evaluation but fail in production.

What Is Data Leakage?

Any situation where the model has access — directly or indirectly — to information it would not have at prediction time. The most common form: fitting preprocessing (normalization, imputation) on the full dataset before splitting.

The Correct Order

Split first, then fit all preprocessing objects on the training set only. Apply the fitted transformations to the test set without refitting. This ensures the test set remains a true simulation of unseen data.

Pipeline as a Safeguard

Encapsulating preprocessing and modeling in a single pipeline object enforces the correct order automatically. When the pipeline is fitted on training data, all internal steps are fitted on training data only.

 Reference: Kaufman, S., Rosset, S., Perlich, C. & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. ACM TKDD, 6(4), 1–21. — Formal treatment of data leakage types and their impact on model evaluation.

Complete Classification Pipeline

This tutorial connects the previous steps into a coherent flow, emphasizing the correct fitting of normalization.

```
library(daltoolbox)

# ... (setup: iris, slevels, stratified split) ...

# Normalization fitted ONLY on training data
norm      <- minmax()
norm      <- fit(norm, iris_train)
iris_train_norm <- transform(norm, iris_train)
iris_test_norm  <- transform(norm, iris_test)

model <- cla_knn("Species", slevels, k = 5)
model <- fit(model, iris_train_norm)

train_eval$metrics # accuracy=1.000
test_eval$metrics  # accuracy=0.967
```

- **Split before preprocessing:** The split comes first. Preprocessing before splitting contaminates the test set with information from training data (data leakage).
- **fit() on training data only:** The `minmax()` learns the minimum and maximum from the training data. The same `norm` object is then applied to the test set — without the test set influencing the transformation.
- **Why KNN here?:** KNN is sensitive to scale. Without normalization, attributes with larger ranges dominate the distance calculation. With normalization, all features contribute equally.
- **Result:** Perfect training score (1.000) and test at 0.967 — an expected and healthy difference.

❏ **Key concept:** Never fit preprocessing on the full dataset. Always use only the training data to learn the transformations.

Regression: Predicting Continuous Targets

Regression is the supervised learning task where the target variable is continuous rather than categorical. The experimental workflow — split, fit, predict, evaluate — remains identical to classification; what changes are the metrics used to measure prediction error.

Mean Squared Error (MSE)

Average of squared differences between predicted and observed values. Penalizes large errors more heavily than small ones. Sensitive to outliers in the target variable.

SMAPE

Symmetric Mean Absolute Percentage Error. Measures relative error as a percentage, making it interpretable across datasets with different scales. Symmetric: treats over- and under-prediction equally.

R^2 (Coefficient of Determination)

Proportion of variance in the target explained by the model. $R^2=1$ means perfect prediction; $R^2=0$ means the model is no better than predicting the mean. Negative values indicate the model is worse than the mean.

Train vs. Test Gap

A large gap between training R^2 and test R^2 signals overfitting. Regression trees are particularly prone to overfitting on training data when grown without pruning.

  Reference: Hastie, T., Tibshirani, R. & Friedman, J. (2009). The Elements of Statistical Learning. 2nd ed. Springer.

Regression Workflow

In regression, the flow remains consistent: split, fit, predict, and evaluate the numeric target.

```
library(daltoolbox)
library(MASS)

data(Boston)
Boston <- as.matrix(Boston) # target: medv (continuous)

set.seed(1)
sr <- train_test(sample_random(), Boston)

model <- reg_dtree("medv")
model <- fit(model, sr$train)

train_eval$metrics
# mse=12.68 smape=0.135 R²=0.859

test_eval$metrics
# mse=29.38 smape=0.164 R²=0.512
```

- **Boston dataset:** 506 observations, 13 predictors, target `medv` (median home value). A classic regression problem in the literature.
- **reg_dtree():** Regression tree — same syntax as `cla_dtree()`, but the target is numeric. The framework does not change; only the task type changes.
- **Regression metrics:** MSE (mean squared error), SMAPE (symmetric mean absolute percentage error), and R^2 (explained variance). $R^2=0.859$ on training vs. 0.512 on test indicates moderate overfitting.
- **Interpreting R^2 :** 0.512 means the model explains 51% of the price variance on the test set — reasonable for a simple tree without tuning.

📌 **Key concept:** The workflow is identical to classification. What changes are the metrics: MSE, SMAPE, and R^2 instead of accuracy.

Clustering: Unsupervised Grouping

Clustering is an unsupervised learning task: there is no target variable. The algorithm groups observations based on similarity in the input space. Because there is no ground truth to optimize against, the choice of representation and distance metric is decisive.

k-Means Algorithm

Partitions data into k clusters by minimizing within-cluster variance. Each observation is assigned to the nearest centroid; centroids are updated iteratively until convergence. Sensitive to initialization and scale.

The Role of Preprocessing

k-Means uses Euclidean distance. Attributes with larger ranges dominate the distance calculation. Without normalization, a single high-variance feature can determine the entire clustering structure, regardless of the other features.

Evaluating Without Labels

When true class labels are available (as in iris), clustering entropy measures how pure the resulting clusters are. Without labels, internal indices (silhouette score, Davies-Bouldin) assess cluster compactness and separation.

📖 Reference: Jain, A. K. (2010). Data clustering: 50 years beyond k-means. Pattern Recognition Letters, 31(8), 651–666. — Comprehensive review of clustering algorithms, evaluation criteria, and the challenges of unsupervised learning.

Clustering Workflow

In clustering, preprocessing is decisive: the choice of data representation substantially alters unsupervised results.

```
library(daltoolbox)

x <- iris[, 1:4]

model <- cluster_kmeans(k = 3)
model <- fit(model, x)
clu <- cluster(model, x)
table(clu) # 1=38, 2=50, 3=62
evaluate(model, clu, iris$Species)$clustering_entropy
# 0.394

norm <- fit(minmax(), iris)
x_norm <- transform(norm, iris)[, 1:4]
model_n <- fit(cluster_kmeans(k = 3), x_norm)
clu_n <- cluster(model_n, x_norm)
table(clu_n) # 1=61, 2=50, 3=39
evaluate(model_n, clu_n, iris$Species)$clustering_entropy
# 0.418
```

- **No explicit target:** In clustering, there is no response variable during training. The model groups observations by similarity in the input attributes.
- **cluster():** Returns the cluster index for each observation. `table(clu)` shows how many samples were assigned to each group.
- **Clustering entropy:** Measures the purity of clusters relative to the true classes. Lower entropy = more homogeneous clusters. Here: 0.394 (original) vs. 0.418 (normalized).
- **Effect of normalization:** With the original scale, `Petal.Length` dominates the distance calculation. With normalization, all attributes contribute equally — the clusters change from {38, 50, 62} to {61, 50, 39}.

❏ **Key concept:** Same algorithm, different representation → different clusters. In unsupervised learning, representation matters as much as the method.

Visual Analysis and Reproducible Reporting

Results that cannot be communicated clearly are results that cannot be acted upon. Visual analysis and structured metric reports are not cosmetic additions — they are integral parts of the scientific workflow, enabling both interpretation and reproducibility.

Exploratory Visualization

Scatter plots, histograms, and box plots reveal data structure before modeling begins. Inspecting class separability, feature distributions, and outliers visually often surfaces issues that summary statistics miss.

Metric Tables

A structured table comparing models across the same metrics and split is the standard format for reporting results in data mining. It enables direct, unambiguous comparison and is easy to include in papers or technical reports.

Reproducibility

Reproducible results require fixed random seeds, documented preprocessing steps, and code that can be re-run to produce the same output. Visualization and reporting code should be part of the same script as the modeling code.

  Reference: Wickham, H. (2016). ggplot2: Elegant Graphics for Data Analysis. 2nd ed. Springer. — The definitive reference for the ggplot2 package used in this tutorial for scatter plots and visual reporting.

Visual Analysis and Reports

Visual inspection and metric reports are essential for communicating data mining results.

```
library(daltoolbox); library(ggplot2)
library(RColorBrewer); library(dplyr)

gr <- plot_scatter(
  iris |> dplyr::select(x = Sepal.Length,
    value = Sepal.Width,
    variable = Species),
  label_x = "Sepal.Length",
  label_y = "Sepal.Width",
  colors = brewer.pal(4, "Set1")[1:3]
)
plot(gr)
```

```
models <- list(
  majority = cla_majority("Species", slevels),
  tree = cla_dtree("Species", slevels))
report <- lapply(names(models), function(name) {
  fitted <- fit(models[[name]], sr$train)
  metrics <- evaluate(fitted,
    adjust_class_label(sr$test$Species),
    predict(fitted, sr$test))$metrics
  data.frame(model = name, t(metrics))
})
do.call(rbind, report)
# majority: accuracy=0.333 | tree:
accuracy=0.967
```

- **plot_scatter()**: A daltoolbox function that generates a scatter plot colored by class. Useful for inspecting separability before choosing a classifier — if the classes overlap heavily, expect classification errors.
- **brewer.pal()**: Color palette from RColorBrewer. Set1 provides distinct colors suitable for categorical variables.
- **Comparison table**: `lapply()` + `do.call(rbind, ...)` produces a compact, reproducible table. Ideal for inclusion in reports or articles.
- **Result**: majority=0.333, tree=0.967 — the table communicates the difference clearly and directly.

❏ **Key concept**: Visual analysis and metric tables are not separate steps — they are part of the same workflow for communicating results.

Extending Frameworks with S3 in R

A well-designed framework should be open to extension without requiring modification of its core. In R, the S3 object system enables this through generic functions and method dispatch — allowing new learners to integrate seamlessly into an existing workflow.

Constructor

A function that creates the learner object, stores its configuration (hyperparameters, target attribute, class levels), and assigns the new S3 class. This is the entry point for the framework to recognize the new learner.

`fit()` Method

Implements the training logic for the new class. R dispatches `fit()` to this method when the object's class matches. The method receives training data and returns the fitted object with the trained model stored inside.

`predict()` Method

Implements the prediction logic. Must return predictions in the format expected by the framework's `evaluate()` function. Once implemented, the new learner is used identically to any built-in learner.

 Reference: Chambers, J. M. (2008). *Software for Data Analysis: Programming with R*. Springer. — Chapter 9 covers S3 and S4 object systems in R, including method dispatch and class hierarchies.

Custom Extension

```
library(daltoolbox)

# 1. Construtor
cla_rsnnns_custom <- function(attribute, slevels,
                              size = 5, learn_rate = 0.2, maxit = 100) {
  obj <- daltoolbox::classification(attribute, slevels)
  obj$size <- size; obj$learn_rate <- learn_rate
  obj$maxit <- maxit
  class(obj) <- append("cla_rsnnns_custom", class(obj))
  obj
}

# 2. fit() — chama RSNNs::mlp()
fit.cla_rsnnns_custom <- function(obj, data, ...) {
  # ... ajuste de dados e fatores ...
  obj$model <- RSNNs::mlp(x, y, size = obj$size,
    learnFuncParams = c(obj$learn_rate), maxit =
  obj$maxit)
  obj
}
```

```
# 3. predict() — retorna data.frame com colunas = slevels
predict.cla_rsnnns_custom <- function(object, x, ...) {
  # ... ajuste de x, predict, nomear colunas ...
}

model <- cla_rsnnns_custom("Species", slevels,
  size=5, learn_rate=0.1, maxit=150)
model <- fit(model, sr$train)
eval <- evaluate(model, sr$test$Species,
  predict(model, sr$test))
eval$metrics # accuracy=0.933
```

The daltoolbox allows integrating external methods while maintaining the consistency of the experimental workflow

- **Extension pattern:** Three pieces — constructor, `fit()`, `predict()`. The constructor stores configuration; `fit()` trains; `predict()` returns predictions in the format expected by the framework.
- **classification():** Base function from daltoolbox that initializes the object with the target attribute and class levels. Every custom learner starts here.
- **class(obj) <- append(...):** Adds the new class to the top of the S3 hierarchy. This causes R to dispatch `fit()` and `predict()` to the custom implementations.
- **Identical usage:** After implementing the three functions, the custom learner is used exactly like any built-in — `fit()`, `predict()`, `evaluate()`. Result: accuracy=0.933.

📌 **Key concept:** Extending the framework does not require rewriting the workflow. Simply implement `fit()` and `predict()` for the new class.



Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

